

Démo liens symboliques (en: symbolic links, symlinks, soft links)

Introduction

Les liens symboliques sont l'équivalent des raccourcis sous Windows.

Les liens symboliques sont des fichiers de type 1.

Les liens symboliques sont des fichiers, donc ils possèdent un inode avec des permissions, un compteur de liens (**Nlinks**), un utilisateur propriétaire, un groupe propriétaire, des timestamps, etc.

Le contenu (au sens du filesystem) d'un lien symbolique est un chemin, qui peut être absolu ou relatif. Pensez à un string.

Ce qui différencie un lien symbolique d'un fichier régulier est la façon dont son contenu est interprété.

Lorsque un fichier nommé **<fichier>** est un lien symbolique dont le contenu est **<chemin>**, la plupart des opérations sur **<fichier>** vont s'appliquer au fichier dont le chemin est **<chemin>**. Lorsque **<chemin>** est un chemin relatif, le chemin à partir du répertoire qui nomme **<fichier>**.

L'opération qui consiste à appliquer une commande au fichier qui est nommé par le chemin du lien symbolique, est appelé *déréférencement*.

La syntaxe pour créer un lien symbolique est

```
$ ln -s <chemin> <nom du lien symbolique>
```

Créons un répertoire de test et ajoutons-y un fichier régulier nommé **plop** dont le contenu est la chaîne de caractères **hop** :

```
$ mkdir /tmp/test
$ cd /tmp/test
$ echo hop > plop

$ ls
plop
$ cat plop
hop
```

On peut créer un lien symbolique nommé **bibi** dont le contenu est le chemin **./plop** :

```
$ ln -s ./plop bibi
```

La plupart des commandes appliquées à un lien symbolique vont en fait s'appliquer au fichier pointé par le lien symbolique (c'est le fameux déréférencement):

```
$ cat bibi
hop

$ echo aze >> bibi
$ cat plop
hop
aze
```

Ainsi, la commande **cat** n'affiche pas le contenu du lien symbolique, mais le contenu du fichier pointé par le lien symbolique (si c'est un fichier régulier). On accède au contenu d'un lien symbolique avec la commande **readlink** :

```
$ readlink bibi
./plop
```

Pour les fichiers d'un autre type :

```
$ readlink plop
```

ne renvoie rien car le fichier régulier nommé `plop` n'est pas un lien symbolique, mais il provoque une erreur silencieuse (consulter le man) :

```
$ echo $?  
1
```

Récurtivité du déréférencement

Le chemin pointé par un lien symbolique peut être tout type de fichier, en particulier un répertoire.

```
$ ln -s /etc/apt aptconf  
$ ls aptconf  
apt.conf.d  keyrings      sources.list  trusted.gpg.d  
auth.conf.d preferences.d sources.list.d  
  
$ cat aptconf/sources.list  
deb http://deb.debian.org/debian trixie main  
deb http://deb.debian.org/debian trixie-updates main  
deb http://deb.debian.org/debian-security/ trixie-security main
```

Le chemin pointé par un lien symbolique peut aussi être un lien symbolique, dans ce cas, le déréférencement s'appliquera récursivement :

```
$ ln -s /tmp/test/aptconf aptreconf  
$ ls aptreconf  
apt.conf.d  keyrings      sources.list  trusted.gpg.d  
auth.conf.d preferences.d sources.list.d
```

Et bien sur, on peut combiner à l'infini :

```
$ mkdir -p rep/rap  
$ tree -F  
./  
├── aptconf -> /etc/apt/  
├── aptreconf -> /tmp/test/aptconf/  
├── bibi -> ./plop  
├── plop  
└── rep/  
    └── rap/  
5 directories, 2 files
```

```
$ ln -s ../bibi rep/bobo  
$ ln -s .. rep/rap/parent  
$ tree -F  
./  
├── aptconf -> /etc/apt/  
├── aptreconf -> /tmp/test/aptconf/  
├── bibi -> ./plop  
├── plop  
└── rep/  
    ├── bobo -> ../bibi  
    └── rap/  
        └── parent -> ../  
6 directories, 3 files
```

```
$ cat /tmp/test/rep/rap/parent/bobo
hop
aze
```

Le déréférencement n'est pas systématique

Toutes les commandes ne déréférencent pas.

En particulier, les commandes pour obtenir de l'information sur les liens symboliques ne déréférencent pas. Par exemple, la commande `stat` qui affiche les attributs d'inode d'un fichier s'appliquent sur le lien symbolique :

```
File: bibi -> ./plop
Size: 6          Blocks: 0          IO Block: 4096   symbolic link
Device: 0,38     Inode: 594776       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1004/   alice)   Gid: ( 1004/   alice)
Access: 2025-03-03 14:01:14.453173931 +0100
Modify: 2025-03-03 14:01:12.133226068 +0100
Change: 2025-03-03 14:01:12.133226068 +0100
Birth: 2025-03-03 14:01:12.133226068 +0100
```

Si on désire supprimer un lien symbolique, heureusement que la commande `rm` ne supprimera pas le fichier pointé par le chemin du lien, mais bien le lien lui-même :

```
$ ls -l
total 4
lrwxrwxrwx 1 alice alice  8 Mar  3 14:02 aptconf -> /etc/apt
lrwxrwxrwx 1 alice alice 17 Mar  3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 alice alice  6 Mar  3 14:01 bibi -> ./plop
-rw-r--r-- 1 alice alice  8 Mar  3 14:01 plop
drwxr-xr-x 3 alice alice 80 Mar  3 14:02 rep

$ rm aptconf
$ ls -l
total 4
lrwxrwxrwx 1 alice alice 17 Mar  3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 alice alice  6 Mar  3 14:01 bibi -> ./plop
-rw-r--r-- 1 alice alice  8 Mar  3 14:01 plop
drwxr-xr-x 3 alice alice 80 Mar  3 14:02 rep
```

Forcer à ne pas déréférencer

Il peut arriver qu'on veuille appliquer une commande au lien symbolique lui-même plutôt qu'au fichier pointé par le chemin de son contenu.

Pour cela, certaines commandes ont une option `--no-dereference` pour signifier que la commande doit agir sur le lien symbolique plutôt que sur le fichier pointé par son contenu.

Par exemple, si on exécute

```
$ sudo chown root bibi
$ ls -l
total 4
lrwxrwxrwx 1 alice alice 17 Mar  3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 alice alice  6 Mar  3 14:01 bibi -> ./plop
-rw-r--r-- 1 root  alice  8 Mar  3 14:01 plop
drwxr-xr-x 3 alice alice 80 Mar  3 14:02 rep
```

on voit que le changement d'utilisateur propriétaire s'est appliqué au fichier nommé **plop**, mais si on exécute

```
$ sudo chown --no-dereference bob bibi
$ ls -l
total 4
lrwxrwxrwx 1 alice alice 17 Mar  3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 bob   alice  6 Mar  3 14:01 bibi -> ./plop
-rw-r--r-- 1 root  alice  8 Mar  3 14:01 plop
drwxr-xr-x 3 alice alice 80 Mar  3 14:02 rep
```

on voit que le changement d'utilisateur propriétaire s'est appliqué au lien symbolique lui-même.

Inversement, il existe aussi une option **--dereference** forcer le déréférencement pour les commandes qui considèrent les liens tels quels sans déréférencer :

```
$ stat bibi
  File: bibi -> ./plop
  Size: 6          Blocks: 0          IO Block: 4096   symbolic link
Device: 0,38      Inode: 594776       Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1005/   bob)   Gid: ( 1004/   alice)
Access: 2025-03-03 14:05:25.703527535 +0100
Modify: 2025-03-03 14:01:12.133226068 +0100
Change: 2025-03-03 14:05:22.751593875 +0100
Birth: 2025-03-03 14:01:12.133226068 +0100

$ stat --dereference bibi
  File: bibi
  Size: 8          Blocks: 8          IO Block: 4096   regular file
Device: 0,38      Inode: 594775       Links: 1
Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: ( 1004/   alice)
Access: 2025-03-03 14:01:33.720740926 +0100
Modify: 2025-03-03 14:01:27.736875403 +0100
Change: 2025-03-03 14:04:48.864355429 +0100
Birth: 2025-03-03 14:00:41.877906000 +0100
```

Notez en particulier la différence de type et de numéro d'inode et la correspondance:

```
$ ls -il
total 4
594778 lrwxrwxrwx 1 alice alice 17 3 mars 14:02 aptreconf -> /tmp/test/aptconf
594776 lrwxrwxrwx 1 bob   alice  6 3 mars 14:01 bibi -> ./plop
594775 -rw-r--r-- 1 root  alice  8 3 mars 14:01 plop
594779 drwxr-xr-x 3 alice alice 80 3 mars 14:02 rep
```

Certaines commandes permettent d'être explicite dans les deux cas :

```
$ cp --dereference bibi bobo
$ ls -l
total 8
lrwxrwxrwx 1 alice alice 17 Mar  3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 bob   alice  6 Mar  3 14:01 bibi -> ./plop
-rw-r--r-- 1 alice alice  8 Mar  3 14:09 bobo
-rw-r--r-- 1 root  alice  8 Mar  3 14:01 plop
drwxr-xr-x 3 alice alice 80 Mar  3 14:02 rep

$ cp --no-dereference bibi baba
$ ls -l
total 8
lrwxrwxrwx 1 alice alice 17 Mar  3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 alice alice  6 Mar  3 14:09 baba -> ./plop
lrwxrwxrwx 1 bob   alice  6 Mar  3 14:01 bibi -> ./plop
```

```
-rw-r--r-- 1 alice alice 8 Mar 3 14:09 bobo
-rw-r--r-- 1 root  alice 8 Mar 3 14:01 plop
drwxr-xr-x 3 alice alice 80 Mar 3 14:02 rep
```

Le fichier `bobo` est un fichier régulier qui est une copie du fichier `plop` (son contenu est la chaîne de caractères `hop\naze\n`) alors que le fichier `bibi` est un lien symbolique qui est une copie du fichier `bobo` (son contenu est le chemin `./plop`).

Liens cassés

Remarque importante : il n'est pas nécessaire qu'un fichier nommé par le chemin d'un lien symbolique existe pour créer le lien symbolique :

On peut tout à fait exécuter la commande :

```
$ ln -s ./aze qsd
```

Cela va créer le lien symbolique nommé `qsd`, bien qu'aucun fichier n'est nommé `aze` par le répertoire courant.

```
$ cat qsd
cat: qsd: Aucun fichier ou dossier de ce type

$ ls -l
total 8
lrwxrwxrwx 1 alice alice 17 Mar 3 14:02 aptreconf -> /tmp/test/aptconf
lrwxrwxrwx 1 alice alice 6 Mar 3 14:09 baba -> ./plop
lrwxrwxrwx 1 bob  alice 6 Mar 3 14:01 bibi -> ./plop
-rw-r--r-- 1 alice alice 8 Mar 3 14:09 bobo
-rw-r--r-- 1 root  alice 8 Mar 3 14:01 plop
lrwxrwxrwx 1 alice alice 5 Mar 3 14:11 qsd -> ./aze
drwxr-xr-x 3 alice alice 80 Mar 3 14:02 rep
```

On appelle cela un « lien cassé » (en: broken link) ou « lien mort » (en: dead link, deadlink).

Bien sûr, si on crée un fichier nommé `aze` dans le répertoire courant, le lien symbolique jouera son rôle :

```
$ echo hop > aze
$ cat qsd
hop
```

Remarque sur l'ordre des arguments

On pourrait argumenter sur le fait qu'une syntaxe du type

```
$ ln -s <nom du lien symbolique> <chemin>
```

serait plus naturelle ou plus intuitive. La syntaxe

```
$ ln -s <chemin> <nom du lien symbolique>
```

garde la cohérence avec la copie de fichiers : quand on exécute

```
$ cp <source> <destination>
```

c'est bien le fichier `<source>` qui contient de l'information et le fichier `<destination>` qui va être créé en récupérant l'information contenue dans le fichier `<source>`. De même pour `cp -l`.

Utilisations

Comme on l'a vu, les liens symboliques sont utilisés comme raccourcis, par exemple pour « rapprocher » des fichiers éloignés dans l'arborescence.

Les liens symboliques sont aussi utilisés comme « interrupteurs » pour sélectionner des configurations activées parmi un ensemble de configurations disponibles. Voir par exemple le fonctionnement de `/etc/nginx/sites-enabled` avec `/etc/nginx/sites-available` dans le TP de découverte de `nginx`.